

Coal Rake-Mix Optimizer — Project Analysis & Progress Report

1. Executive summary

The **Coal Rake-Mix Optimizer** (UPRVUNL) is a three-part Python/JS project that recommends how to redistribute monthly coal-rake allocations across plants and coal companies to reduce the blended Variable Charge (VC) toward a ₹4.00/unit target. It consists of a CLI (`main.py`), a self-contained dashboard (`output/dashboard.html`), and an OR-Tools optimization backend (`backend/`). A full architectural breakdown was produced separately as `summary.md` / `summary_KT.pdf` .

This report covers everything done **in this session** to get the project running locally: environment setup, the problems hit, how each was diagnosed and fixed, verification performed, and the current state of each component as of now.

Bottom line: all three components are proven working. The one item needing attention before the OR-Tools button will respond again is that **Docker Desktop is not currently running** (see §5) — the container needs to be restarted.

2. Scope of work performed

#	TASK	STATUS
1	Analyze project structure and purpose	✓ Done
2	Set up local Python environment (venv) for the CLI	✓ Done
3	Diagnose and fix a native-library load failure blocking <code>pandas /CLI</code>	✓ Fixed
4	Run the CLI end-to-end and regenerate the dashboard from <code>input.xlsx</code>	✓ Done, verified
5	Diagnose why the OR-Tools backend couldn't start natively on this machine	✓ Root-caused
6	Containerize the backend (Dockerfile) to bypass the OS restriction	✓ Done
7	Verify the backend end-to-end (<code>/health</code> , <code>/optimize</code> with real data, CORS)	✓ Verified
8	Diagnose dashboard-not-loading / button-error reports	✓ Resolved (user-side navigation issues, not project bugs)
9	Produce a Knowledge Transfer document (<code>summary.md</code>)	✓ Done
10	Render the KT document to PDF (<code>summary_KT.pdf</code>)	✓ Done, content-verified
11	This progress report	✓ Done

3. Issues found, root cause, and fix

3.1 CLI crashed on `import pandas`

- **Symptom:** `ImportError: DLL load failed while importing np_datetime: An Application Control policy has blocked this file.`
- **Root cause:** Windows **Smart App Control** (confirmed ON via `Get-MpComputerStatus`) blocked the native compiled component of the freshly-installed `pandas 3.0.5` wheel.
- **Fix:** Pinned `pandas==2.2.3` in the project's `.venv` — an older, already-trusted build that Smart App Control allows.
- **Verification:** `python main.py --dashboard dashboard_template.html` ran to completion, reproduced the Excel's known-correct overall weighted VC (`4.022564` , matching the sheet), and regenerated `output/dashboard.html` + `output/output.xlsx` .

3.2 OR-Tools blocked entirely, backend can't start natively

- **Symptom:** `OSError: [WinError 4551] An Application Control policy has blocked this file` when loading `ortools` 's native DLL — same root cause as §3.1, but OR-Tools has no older/alternate build that

avoids it since it's a compiled solver (SCIP), not a pure-Python fallback situation.

- **Impact:** `backend/optimizer.py` imports OR-Tools at module level, so `uvicorn main:app` would crash immediately on this machine if run natively. The CLI degrades gracefully (falls back to its own greedy heuristic per plant), but the backend has no such fallback by design — its entire value is being the "real" OR-Tools solver.
- **Fix (agreed with you rather than disabling a system security feature):** Wrote `backend/Dockerfile` + `backend/.dockerignore` and built/ran the backend inside a **Linux container** via the already-installed Docker Desktop. Smart App Control does not apply inside the container, so OR-Tools loads normally there.
- **Verification:**
 - `GET /health` → `{"status": "ok"}`
 - `POST /optimize` fed with real data derived from `input.xlsx` → `status: "Optimal"`, weighted VC improved **4.0226** → **3.9429**, 15 diversions proposed, all 5 constraints (company conservation, plant total bounds, plant-company bounds, global conservation, integer allocation) reported satisfied.
 - CORS preflight for `Origin: null` (how a `file://`-opened dashboard identifies itself) returns `200 OK` with the right `access-control-allow-origin` header — confirms the dashboard can call this backend directly from disk, no local web server needed.

3.3 Dashboard appeared not to open / "Method Not Allowed" confusion

- **Symptom (reported):** dashboard tab looked blank/black; clicking "Optimize Allocation (OR-Tools)" seemed to show `Method Not Allowed`.
- **Root cause:** Not a project bug. Two separate mix-ups: 1. GUI-launch commands (`start` , `Start-Process`) issued from this tool's automation session don't surface a window on your **visible** interactive desktop — they run in a different Windows session, so the tab "opened" but you never saw it. Fixed by having you open the file yourself via `file:///C:/Users/itisa/.../output/dashboard.html`, which rendered correctly with full data. 2. `{"detail": "Method Not Allowed"}` came from navigating **directly** to `http://localhost:8000/optimize` in the address bar (a GET request) — that endpoint is POST-only and is never meant to be visited directly; it's only called in the background by the dashboard's JS when you click the button. Backend logs confirmed no POST request was ever made from an actual button click at the time of the report.
- **Resolution:** Clarified the correct way to use the dashboard; no code changes were needed.

4. What was verified to actually work (not just "should work")

COMPONENT	VERIFIED HOW	RESULT
CLI Excel parsing	Ran <code>main.py</code> , compared to the sheet's known VC	Matches (<code>4.022564102564102</code>)
CLI per-plant optimizer	Ran end-to-end for all 5 plants	Produced improved VC for 4/5 plants vs. baseline
Dashboard data refresh	<code>--dashboard</code> flag regenerates <code>output/dashboard.html</code>	5 plants, 18 plant-source rows embedded correctly
Dashboard rendering	Opened <code>file:///.../dashboard.html</code> in your browser	Confirmed via screenshot: KPI strip, plant cards, tables all rendering
Backend health	<code>curl http://localhost:8000/health</code>	<code>{"status":"ok"}</code>
Backend optimizer (real solve)	Scripted <code>POST /optimize</code> with data derived from <code>input.xlsx</code>	Optimal, VC <code>4.0226</code> → <code>3.9429</code> , 15 diversions, all constraints satisfied
Backend CORS for dashboard	<code>OPTIONS /optimize</code> with <code>Origin: null</code>	<code>200 OK</code> , correct CORS headers

5. Current state as of this report

- **CLI:** Working, `.venv` has `pandas==2.2.3`, `openpyxl`, `ortools` (native `ortools` still blocked locally, so the CLI's own optimizer transparently uses its greedy fallback — this is expected and by design).
- **Dashboard:** `output/dashboard.html` is up to date with `input.xlsx` as of the last `--dashboard` regeneration.
- **Backend:** ⚠️ **Docker Desktop is not currently running** on this machine (`docker ps` fails to reach the Docker Engine). The `coal-optimizer-backend` image was built and previously verified working, but the container is not live right now. To bring it back up: `bash # 1. Start Docker Desktop (from the Start menu), then: docker start coal-optimizer-backend # or, if the container was removed: cd backend && docker build -t coal-optimizer-backend . docker run -d --name coal-optimizer-backend -p 8000:8000 coal-optimizer-backend`

6. Deliverables produced this session

FILE	PURPOSE
<code>.venv/</code>	Local Python environment for the CLI (pandas 2.2.3, openpyxl, ortools, fastapi, uvicorn, pydantic)
<code>backend/Dockerfile</code>	Builds a Linux image for the FastAPI + OR-Tools backend
<code>backend/.dockerignore</code>	Keeps the Docker build context clean
<code>output/dashboard.html</code>	Regenerated with live data from <code>input.xlsx</code>
<code>output/output.xlsx</code>	CLI's optimization result export
<code>summary.md</code>	Full architectural Knowledge Transfer document
<code>summary_KT.pdf</code>	PDF rendering of the above (9 pages, content-verified)
<code>PROGRESS_REPORT.md</code>	This report

7. Recommendations / next steps

1. **Restart Docker Desktop** and bring the backend container back up before relying on the "Optimize Allocation (OR-Tools)" button again.
2. Consider asking your IT/security team whether the specific `ortools` and `pandas` binaries can be added to an allow-list, if running everything natively (without Docker) is a longer-term requirement.
3. If this needs to run reliably on other machines, the Docker path for the backend is the most portable option since it sidesteps this specific Windows restriction entirely.
4. No functional bugs were found in the project's own code — every issue traced back to this machine's security policy or to navigation mix-ups, not the optimizer logic itself.